

RAPPORT D'AUDIT EXPRESSION ARENA

Audit complet — Sécurité, RGPD, Qualité, Comportement
Généré le 15 May 2026 · Confidentiel

Domaine	Findings	Critique	Élevé	Moyen
Sécurité	8	3	3	2
RGPD	6	1	3	2
Qualité de code	7	0	2	5
Comportement	7	2	3	2
TOTAL	28	6	11	11

Sommaire

1. Sécurité	3
2. RGPD & Conformité	5
3. Qualité de code	7
4. Comportement / UX	9
5. Plan d'action	11

1. Sécurité

Expression Arena s'appuie sur Supabase (Postgres + RLS) et des cookies de session pour authentifier les joueurs. Plusieurs failles permettent à un attaquant de se faire passer pour n'importe quel joueur, d'accéder à des données privées ou de manipuler l'état de jeu côté serveur.

1.1 Authentification & Sessions

Sév.	Problème	Détail / Impact
CRITIQUE	Cookie de session non signé	Le cookie 'session' contient un UUID en clair. N'importe qui peut forger n'importe quelle valeur pour usurper l'identité d'un joueur. FIX : signer avec iron-session ou jose (JWT HS256).
CRITIQUE	Pas de rate-limiting sur /login et /register	Brute-force illimité possible sur le mot de passe. FIX : middleware avec upstash/ratelimit ou vercel edge rate limit.
ÉLEVÉ	Pas d'expiration/rotation de session	Un cookie volé est valide indéfiniment (pas de maxAge défensif, pas de refresh). FIX : rotation à chaque requête + expiration 24h.

1.2 Row-Level Security (RLS)

Toutes les politiques SELECT existantes utilisent USING(true), ce qui expose toutes les lignes à tous les utilisateurs authentifiés — y compris les messages privés d'autres utilisateurs, les infos de profil, les résultats de parties.

Sév.	Problème	Détail / Impact
CRITIQUE	RLS USING(true) sur direct_messages	Tout joueur authentifié peut lire TOUS les DMs via l'API Supabase directe. FIX : USING(auth.uid() = sender_id OR auth.uid() = recipient_id).
ÉLEVÉ	RLS USING(true) sur players	Les champs email_hash, avatar_url, tous les scores sont lisibles par n'importe qui. FIX : séparer profil public / données privées ou restreindre.
ÉLEVÉ	Pas de RLS sur presence	La table presence n'a aucune politique UPDATE/DELETE. N'importe qui peut modifier le statut d'un autre joueur. FIX : USING(auth.uid())::text = player_id).
MOYEN	Absence de CSRF protection sur les server actions	Les actions /api/presence et /api/leave ne vérifient pas l'origine. FIX : vérifier le header Origin ou utiliser un token CSRF.

1.3 Upload d'avatars

Sév.	Problème	Détail / Impact
MOYEN	Pas de validation magic bytes sur l'upload d'avatar	Seul le MIME type déclaré par le client est vérifié — facilement falsifiable. Un attaquant peut uploader un fichier malveillant (HTML, SVG avec script, etc.). FIX : lire les 12 premiers octets et vérifier la signature binaire (JPEG: FFD8FF, PNG: 89504E47...).

2. RGPD & Conformité

L'application collecte des données personnelles (pseudo, email de contact, avatar, historique de parties) et héberge des communications privées. Elle est donc soumise au RGPD. Plusieurs non-conformités ont été identifiées.

2.1 Politique de confidentialité

Sév.	Problème	Détail / Impact
ÉLEVÉ	Cookie présenté comme durant '24h' alors qu'il dure 30 jours	La politique de confidentialité affiche une durée de conservation incorrecte. Risque d'amende CNIL. FIX : corriger la durée ou adapter le cookie maxAge.
ÉLEVÉ	Mention 'aucun email collecté' factuellement fausse	La page /contact enregistre les emails en base. La politique dit l'inverse. FIX : mettre à jour la politique ou anonymiser les emails du formulaire contact.
MOYEN	Nom du directeur de publication absent des mentions légales	Obligation légale (LCEN art. 6). FIX : ajouter le nom du responsable éditorial.

2.2 Droits des personnes

Sév.	Problème	Détail / Impact
CRITIQUE	deleteAccount() ne purge pas toutes les données	La suppression de compte n'efface pas : les messages directs envoyés, les avatars dans le storage Supabase, les abonnements push. FIX : cascade DELETE sur direct_messages (sender_id), delete avatar from bucket, unsubscribe push tokens.
ÉLEVÉ	Pas de consentement explicite à l'inscription	Aucune case CGU/Politique de confidentialité n'est affichée sur /register. Obligation RGPD art. 7. FIX : ajouter checkbox obligatoire avec lien.
MOYEN	Footer absent des pages de jeu	Les pages /play/*, /result, /room/* n'ont pas de footer avec liens légaux. FIX : inclure le footer dans le GameLayout ou ajouter un mini-footer.

3. Qualité de code

L'audit a analysé l'ensemble des 14 fichiers critiques du projet. La duplication de code est le problème structurel dominant, rendant la maintenance coûteuse et les bugs difficiles à corriger de façon cohérente.

3.1 Duplication massive

Sév.	Problème	Détail / Impact
ÉLEVÉ	updateLeaderboard() dupliquée x14	Chaque fichier /play/*Client.tsx contient sa propre copie de la logique de mise à jour du classement. Un bug corrigé dans une copie ne l'est pas dans les autres. FIX : extraire dans src/lib/leaderboard.ts et importer partout.
ÉLEVÉ	GAME_LABELS dupliqué x7	Le mapping game_type -> libellé lisible est redéfini dans 7 fichiers différents. FIX : src/lib/games.ts avec export const GAME_LABELS.
MOYEN	handleSave/handleMove dupliqué dans Naval/Mastermind/Reflexe	La logique de validation et de soumission de coup est similaire dans plusieurs jeux. Opportunité de hook partagé useGameTurn().

3.2 Bugs silencieux

Sév.	Problème	Détail / Impact
MOYEN	Morpion : myMark jamais utilisé après assignation	myMark est calculé mais la logique utilise myId directement pour déterminer qui joue — le bug ne se voit pas mais crée une incohérence subtile.
MOYEN	ResultClient : cast GameType incorrect	Le composant caste game_type as GameType mais le type n'inclut pas tous les types possibles en DB — risque de undefined silent dans les libellés.
MOYEN	game_settings (win/draw/loss_pts) jamais lus par les jeux	Les paramètres configurés dans l'admin ne sont jamais utilisés pour calculer les points distribués. Les jeux utilisent des valeurs hardcodées. FIX : fetchGameSettings() dans chaque Client au mount et appliquer.
MOYEN	0 tests automatisés	Aucun fichier *.test.* dans tout le projet. Les régressions ne sont détectées qu'en production. FIX : commencer par les utils purs (leaderboard, gameLabels).

4. Comportement & UX

Cette section couvre les bugs fonctionnels qui affectent l'expérience de jeu et les comportements non désirés détectés lors de l'audit.

4.1 Triche possible

Sév.	Problème	Détail / Impact
CRITIQUE	Naval/Mastermind/Plus-ou-Moins : état transmis en clair via Realtime	L'état de jeu complet (position des bateaux, code Mastermind, nombre cible) est transmis en realtime à TOUS les joueurs. Un joueur peut ouvrir la console et lire les données adverses. FIX : ne transmettre que les coups et résultats, jamais l'état privé.
CRITIQUE	Reflexe : timestamp côté client	Le temps de réaction est calculé côté client et envoyé tel quel au serveur. Un joueur peut envoyer n'importe quelle valeur. FIX : timestamper côté serveur au moment de la réception du coup.

4.2 Chat

Sév.	Problème	Détail / Impact
ÉLEVÉ	Les utilisateurs bloqués ne sont pas filtrés dans ChatSystem	La table 'blocks' existe mais ChatSystem ne filtre pas les messages des joueurs bloqués. FIX : charger la liste des blocks au mount et filtrer les messages.
MOYEN	Compteur de messages non lus plafonné à 1	Le badge du chat n'affiche que 0 ou 1 — jamais le vrai nombre. FIX : incrémenter un compteur à chaque message entrant non lu.
MOYEN	Abonnement push non supprimé à la déconnexion	Quand un joueur se déconnecte, son push token reste en base. Il peut recevoir des notifs après logout. FIX : DELETE push_subscriptions WHERE player_id=... au logout.

4.3 UX & Navigation

Sév.	Problème	Détail / Impact
MOYEN	Pas de page 404 ou 500 personnalisée	Next.js affiche ses pages d'erreur génériques. FIX : créer app/not-found.tsx et app/error.tsx avec le design du projet.
FAIBLE	Absence de skeleton/loading sur les pages de jeu	La transition vers /play/* affiche un écran blanc pendant le chargement. FIX : app/(game)/play/loading.tsx avec un skeleton.

5. Plan d'action

Les findings sont regroupés en 5 phases selon leur urgence et leur impact. Les phases 1 et 2 sont prioritaires avant tout déploiement public.

Phase 1 — Sécurité critique

À traiter AVANT toute mise en production publique.

- Signer les cookies de session (iron-session / jose)
- Corriger les politiques RLS sur direct_messages, presence
- Ajouter le rate-limiting sur /login et /register
- Valider les magic bytes lors de l'upload d'avatar

Phase 2 — Bugs fonctionnels critiques

Avant l'ouverture à de nouveaux joueurs.

- Corriger la transmission d'état en clair (Naval, Mastermind, Plus-ou-Moins)
- Timestamper les coups Reflexe côté serveur
- Brancher game_settings sur les calculs de points
- Filtrer les utilisateurs bloqués dans ChatSystem

Phase 3 — RGPD & Légal

Délai : 1 mois maximum (risque CNIL).

- Corriger la politique de confidentialité (durée cookie, emails)
- Ajouter le nom du directeur de publication
- Compléter deleteAccount() (messages, avatar, push)
- Ajouter le consentement CGU à l'inscription
- Ajouter le footer légal aux pages de jeu

Phase 4 — Dette technique

Améliore la maintenabilité à long terme.

- Extraire updateLeaderboard() dans src/lib/leaderboard.ts
- Centraliser GAME_LABELS dans src/lib/games.ts
- Corriger le bug myMark dans Morpion
- Corriger le cast GameType dans ResultClient
- Ajouter les premiers tests unitaires (leaderboard, games utils)

Phase 5 — UX

Améliore la qualité perçue.

- Pages 404 et 500 personnalisées
- Skeleton loading sur les pages de jeu
- Compteur de messages non lus correct
- Supprimer le push token au logout

Ce rapport a été généré à partir d'un audit automatique du code source. Il ne remplace pas un pentest professionnel. Certaines vulnérabilités peuvent nécessiter une analyse manuelle approfondie.